KIT - KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY

COIMBATORE

(An Autonomous Institution)

IoT - DNN PROJECT REPORT

IoT-BASED VEHICLE ACCIDENT DETECTION AND ALERT SYSTEM

NAME	MUKESH S
REG.NO	711524BCS095
CLASS & SECTION	III CSE A
SUBJECT	IOT & DNN
TITLE	IoT-Based Vehicle Accident Detection and Alert System

ABSTRACT

Road accidents require fast detection and dependable communication of a vehicle's location. This project presents a simulation-first Internet of Things system that models an ESP32 with an MPU6050 accelerometer and gyroscope, buzzer and driver cancel button. Sensor readings are organized into time windows and sent using MQTT to a Node.js backend. The backend screens telemetry, computes motion features and requests an accident classification from a Python FastAPI service. Incident state is stored in MongoDB and live status is shown in a React dashboard through Socket.IO.

A Wokwi circuit demonstrates the ESP32 and sensor behavior. Location is represented by fixed fallback coordinates; a GPS receiver is not simulated. The project repository contains simulator-generated evaluation data and saved metric reports. Code inspection shows that the current final prediction path uses threshold rules, although artifacts use a DNN label. Consequently, saved metrics are not a field validation or proof of a trained neural network. The Wokwi-to-local-broker integration additionally requires the Wokwi Private IoT Gateway, which was unavailable during integration verification.

1. INTRODUCTION

When an accident occurs, a vehicle may be unable to report the event. An embedded sensing unit can monitor acceleration and rotation, detect an impact-like pattern and pass an alert to a monitoring application. A useful system must distinguish accidents from ordinary events such as hard braking, sharp turns and potholes, and give the driver a short opportunity to cancel a false alarm.

1.1 Problem Statement

The project aims to create an affordable prototype that observes vehicle motion, recognizes abnormal impact patterns, attaches a location field and provides an alert workflow. The design combines sensor simulation, message transport, backend classification, incident handling and a browser dashboard.

1.2 Objectives

- Simulate an ESP32 and MPU6050-based sensing unit in Wokwi.
- Transmit status, telemetry and event messages using MQTT.
- Screen rolling sensor data and evaluate accident candidates.
- Provide buzzer activation and a driver cancellation mechanism.
- Display device status and incident information on a real-time dashboard.

2. SYSTEM REQUIREMENTS AND SCOPE

2.1 Functional Requirements

ID	Requirement	System response
FR1	Read motion data.	Read accelerometer and gyroscope samples from MPU6050.
FR2	Transmit telemetry.	Publish device samples and status using MQTT.
FR3	Recognize candidate impacts.	Buffer samples, derive window features and screen for motion changes.
FR4	Manage alerts.	Create pending incidents, start a countdown and allow cancellation.
FR5	Monitor fleet.	Show live device readings and incident events in the web dashboard.

2.2 Hardware Requirements

Component	Purpose
ESP32 DevKit	Embedded controller with Wi-Fi and I2C interface.
MPU6050	Three-axis accelerometer and gyroscope for motion sensing.
Buzzer	Local audible alert output.
Push button	Driver cancellation input.
GPS receiver	Planned real-world location source; Wokwi uses fallback coordinates.

2.3 Software Requirements

- Wokwi simulator and ArduinoJson / PubSubClient libraries for the simulated ESP32.
- Eclipse Mosquitto MQTT broker and MongoDB storage.
- Node.js backend with Express, MQTT subscriber and Socket.IO.
- Python FastAPI ML service and React/Vite dashboard.
- Docker Desktop for local broker and database setup.

2.4 Scope and Assumptions

The prototype is intended for educational simulation and software integration. It is not a certified vehicle safety system and has not been validated using real road accidents. It assumes timestamped sensor data, a reachable MQTT broker and configured backend services. The Wokwi circuit does not simulate GPS acquisition or actual vehicle dynamics.

3. SYSTEM ARCHITECTURE

The application is divided into sensing, communication, processing, storage and presentation layers. The device or virtual fleet simulator publishes data to the MQTT broker. The backend consumes telemetry, maintains a per-device sliding window, invokes classification for candidate events and stores incident records. Socket.IO pushes status and alert events to the React client.

Layer	Technology	Role
Device	ESP32 / Wokwi MPU6050	Acquire motion values and control buzzer/button.
Transport	Mosquitto MQTT	Carry telemetry, status, events and commands.
Backend	Node.js / Express	Ingestion, feature windows, incidents and API.
ML	Python / FastAPI	Prediction endpoint, training workflow and metrics.
Storage	MongoDB	Persist telemetry, incidents, users and devices.
Presentation	React / Socket.IO	Live fleet monitoring and incident views.

3.1 Data Flow

1. The ESP32 reads MPU6050 axes and assembles a JSON telemetry sample.
2. The device publishes to vehicle/{deviceId}/telemetry through MQTT.
3. The backend subscriber validates and buffers samples for that device.
4. A feature extractor computes acceleration, jerk, speed and gyro statistics.
5. A screened candidate is sent to the ML service; accepted incidents enter a timed confirmation state.
6. MongoDB stores records and Socket.IO notifies the dashboard.

3.2 Local Integration Requirement

The Wokwi sketch uses host.wokwi.internal:1883 to reach the developer's local broker through Wokwi's Private IoT Gateway. The public Wokwi gateway cannot reach local services. Without the private gateway, Wokwi hardware can still be simulated, but MQTT traffic does not enter the local backend.

4. HARDWARE DESIGN AND IMPLEMENTATION

4.1 Circuit Connections

Device pin	Connection
ESP32 3V3	MPU6050 VCC
ESP32 GND	MPU6050 GND and buzzer ground
ESP32 GPIO 21	MPU6050 SDA
ESP32 GPIO 22	MPU6050 SCL
ESP32 GPIO 25	Buzzer signal
ESP32 GPIO 4	Cancel pushbutton input, using pull-up

4.2 Device Firmware

The Wokwi sketch initializes the I2C bus, MPU6050, Wi-Fi and MQTT client. It synchronizes time through NTP before using epoch-millisecond timestamps. The sketch publishes retained device status, telemetry samples at a nominal 50 Hz and event messages. It subscribes to a command topic for buzzer and reset/cancel operations.

4.3 Local Alert Logic

The sketch computes acceleration magnitude from the three axes. When magnitude crosses the impact threshold, an impact event is latched and the buzzer is activated. The cancel button can stop a pending local alert. If the countdown expires, an alert-confirmed event is recorded. When MQTT is unavailable, the local sensing and buzzer logic can still run after time synchronization, while serial output indicates unsent events.

4.4 Location Representation

Wokwi telemetry currently includes fallback latitude 10.9982 and longitude 76.9664, with no GPS fix and zero satellites. These values allow application fields to be demonstrated but do not represent a live receiver or measured vehicle position.

5. SOFTWARE DESIGN AND DETECTION METHOD

5.1 Sliding-Window Processing

The backend maintains a ring buffer for each device. With the default settings, a two-second window is evaluated with partial overlap. A screening step avoids inference for quiet motion and requests prediction when peak acceleration, speed-drop ratio or angular rate indicates a possible event.

5.2 Extracted Features

Category	Features
Acceleration	Peak total G, mean total G, RMS, standard deviation and mean axes.
Dynamics	Maximum jerk in G per second.
Speed	Initial/final speed, speed difference and speed-drop ratio.
Rotation	Maximum and mean gyro magnitude in degrees per second.

Acceleration magnitude is calculated as $\sqrt{(ax^2 + ay^2 + az^2)}$. Jerk approximates the change in acceleration magnitude divided by elapsed time. The Python and Node.js feature implementations are intended to use the same feature definitions.

5.3 Classification and Incident State

The backend ML client calls the FastAPI /predict endpoint and retries once before using a rule-engine fallback if the service cannot be reached. A high-confidence accident candidate creates a pending incident unless a duplicate is active or inside the debounce period. The default pending cancellation countdown is 15 seconds. At expiry, the incident is confirmed; cancellation marks it as a false alarm.

5.4 Important Model Qualification

The source code's current prediction function computes its final label using explicit physical threshold rules. Training reports and API metadata use a DNN label, but this code path does not currently run a learned DNN forward pass for the final class. The autoencoder baseline is also implemented as a simplified fixed projection/reconstruction calculation. The results in this report must therefore be described as saved repository evaluation artifacts, not validated deployed neural-network performance.

6. DATASET AND MODEL EVALUATION

6.1 Dataset

The simulator dataset file contains 20,500 labelled windows, including 2,205 accident-labelled rows and 18,295 non-accident-labelled rows. Synthetic generation models frontal-collision samples and non-accident scenarios such as normal driving, hard braking, sharp turns and potholes. The repository also supports simulator-exported data. This report does not claim that a public vehicle sensor dataset was successfully imported.

6.2 Stored Comparison Results

Saved evaluation artifact	Accuracy	Precision	Recall	F1
DNN-labelled result	99.35%	94.95%	98.69%	96.78%
Autoencoder baseline	96.97%	83.92%	85.57%	84.74%

The DNN-labelled saved confusion matrix is TN=2,777, FP=16, FN=4, TP=301. The autoencoder saved matrix is TN=2,743, FP=50, FN=44, TP=261. These figures come from the repository's model_comparison.json and are not independently reproduced here.

6.3 Domain Shift and Noise

3,000-sample domain-shift record	Accuracy	Precision	Recall	F1
DNN-labelled	96.73%	80.69%	97.64%	88.36%
Rule engine	98.27%	100.00%	86.35%	92.68%

The saved 3× noise result reports 28.15% accuracy, 12.05% precision and 100.00% recall. The false-positive count is 2,226, indicating poor robustness to that perturbation. MOMENT was not evaluated because the PyTorch runtime could not load under the current Windows application-control policy.

7. APPLICATION INTEGRATION

7.1 Backend Services

The backend subscribes to vehicle telemetry, status and event topic patterns. Telemetry is queued for persistence and pushed into the sliding-window service. Status updates maintain device presence. Firmware events are forwarded to the web application, and an alert-cancelled event requests cancellation of a pending incident.

7.2 Dashboard

The React application includes a Fleet Monitor that displays device online state, speed, acceleration magnitude, axis values, route and location when available. Socket.IO events provide live telemetry and device status updates. Incident views support pending, confirmed and cancelled event workflows.

7.3 MQTT Message Topics

Topic	Direction	Message purpose
vehicle/{id}/status	Device to backend	Online state, route, plate and timestamp.
vehicle/{id}/telemetry	Device to backend	Sensor window samples with accel, gyro and GPS fields.
vehicle/{id}/event	Device to backend	Impact, confirmation or cancellation event.
vehicle/{id}/command	Backend to device	Buzzer ON/OFF or CANCEL/RESET operation.

7.4 Local Integration Status

Previous local checks showed HTTP 200 responses from the backend, ML service, simulator and frontend. The local MQTT broker was listening on port 1883. A local MQTT-to-backend-to-Socket.IO test and an ML normal-window prediction were also previously verified. The private Wokwi gateway was not available, so a live Wokwi-to-local-broker incident and dashboard path remains unverified.

7.5 Privacy and Security

The repository's anonymous Mosquitto configuration is for local development only. It must not be exposed publicly. A deployment should use device authentication, per-device topic permissions, TLS, secret management, network isolation and a defined retention policy for precise location data.

8. TESTING AND RESULTS

Test	Expected observation	Recorded status
Firmware startup	READY and time synchronization.	Observed in prior Wokwi run.
Sensor update	MPU6050 sample updates at nominal 50 Hz.	Wokwi simulation exercised.
Local MQTT delivery	Status/telemetry consumed by backend.	Local pipeline previously tested.
ML normal window	NON_ACCIDENT result for normal sample.	Previously returned confidence 0.04.
Impact and cancel	Buzzer/impact event; button cancels pending alert.	Local circuit behavior simulated.
Live Wokwi dashboard	Fleet device and incident appear via local broker.	Not verified; Private Gateway unavailable.

8.1 Findings

- The project services can run locally and the application interfaces are present.
- The Wokwi circuit models the relevant sensing and alert-control interactions.
- The current Wokwi firmware uses fallback coordinates rather than a live GPS receiver.
- The current final prediction logic is threshold based, despite DNN-labelled artifacts.
- The 3× noise evaluation has a high false-positive burden and needs model improvement.
- MOMENT results are unavailable and the live private-gateway integration remains incomplete.

8.2 Result Interpretation

A high saved accuracy on simulator-generated data does not establish safety in real traffic. Accident data are difficult to collect, sensor mounting changes signals, and common driving events can resemble impacts. The system needs independent vehicle-level test splits, calibration, repeated experiments and field-representative event testing before any operational claim.

8.3 Budget Note

The supplied project brief lists an indicative bill of materials of ₹2,550 and a recommended maximum student allocation of ₹5,000, within a stated ₹5,000-₹10,000 target. Actual cost depends on module availability, shared lab equipment, shipping and local vendor prices. The Wokwi simulation avoids requiring every hardware component during software demonstration.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project implements a simulation-first design for IoT vehicle accident monitoring. It connects sensor telemetry concepts with MQTT messaging, backend processing, sliding-window features, incident handling, persistent records and a live dashboard. The Wokwi circuit includes an ESP32, MPU6050, buzzer and cancellation button. The project is suitable as an educational prototype and a foundation for further validation.

The system is not yet a complete real-world accident-alert product. Wokwi access to local services depends on a Private IoT Gateway; GPS remains a fallback value; current inference is rule based rather than a learned DNN path; and model results have not been independently validated on real vehicle data. Those limitations should be resolved before claiming safety or deployment readiness.

9.2 Future Enhancements

7. Implement a genuine trained DNN in the production /predict path and add tests proving that persisted learned weights determine predictions.
8. Validate with a documented, legally usable public or field dataset, split by vehicle/run to avoid data leakage.
9. Replace the simplified autoencoder baseline with a trained normal-only model and evaluate a calibrated threshold.
10. Evaluate MOMENT in a compatible runtime if the project still requires that baseline.
11. Integrate a GPS receiver or realistic GPS simulator with fix quality and stale-location handling.
12. Enable the Wokwi Private IoT Gateway and verify end-to-end impact, cancellation and dashboard workflows.
13. Secure MQTT with TLS, device credentials and topic-level authorization.
14. Measure latency, reliability, false alarms and missed detections under realistic sensor noise.

REFERENCES

15. Wokwi ESP32 Wi-Fi guide, https://docs.wokwi.com/guides/esp32-wifi
16. Eclipse Mosquitto documentation, https://mosquitto.org/documentation/
17. FastAPI documentation, https://fastapi.tiangolo.com/
18. Socket.IO documentation, https://socket.io/docs/v4/
19. Project source files: wokwi/sketch.ino, wokwi/diagram.json, server/src/services/slidingWindowService.js, ml-service/app/features.py and ml-service/app/model.py.
20. Project evaluation artifacts: ml-service/reports/model_comparison.json and ml-service/reports/domain_shift_results.json.

APPENDIX C. WOKWI SIMULATION EVIDENCE

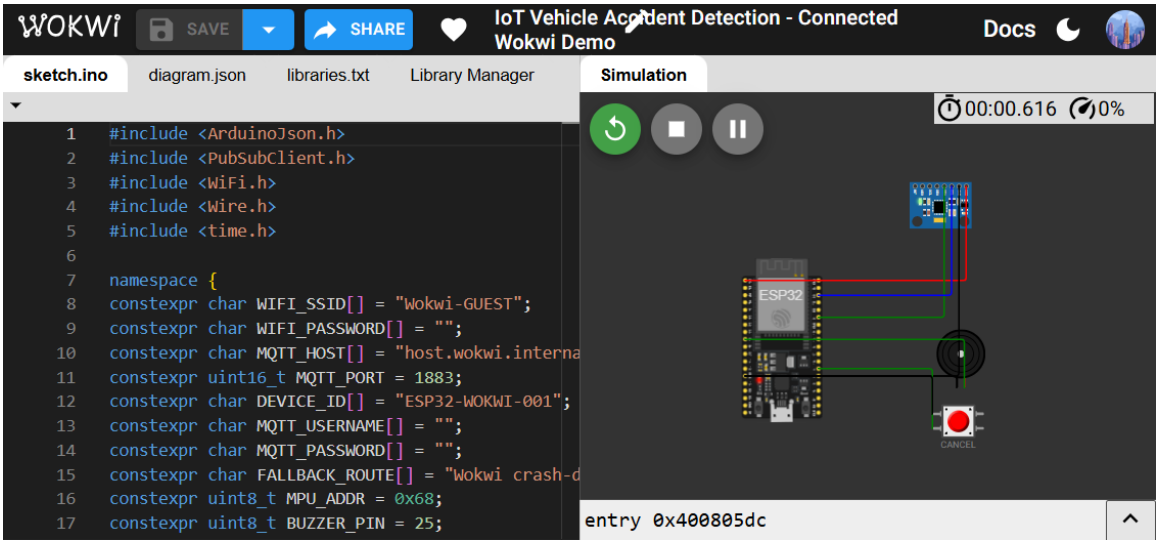


Figure C.1. Wokwi project page showing the ESP32 sketch and connected MPU6050, buzzer and CANCEL button circuit.

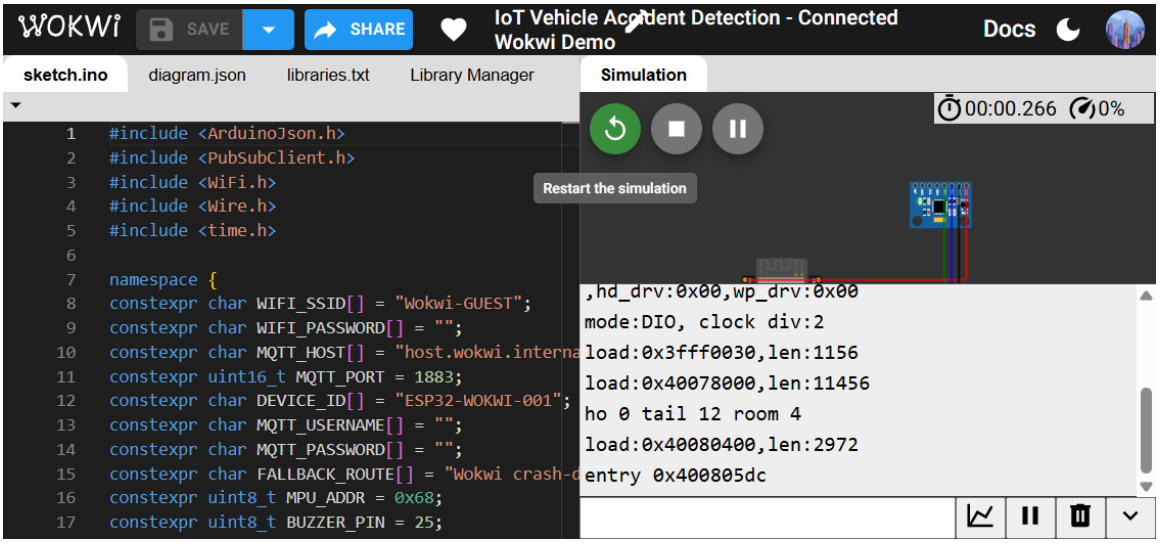


Figure C.2. Serial monitor captured during the simulation launch. This capture shows the ESP32 ROM boot output only; READY and MQTT_CONNECTED were not observed, so it does not prove a successful firmware or backend connection.

APPENDIX D. APPLICATION AND EVALUATION EVIDENCE

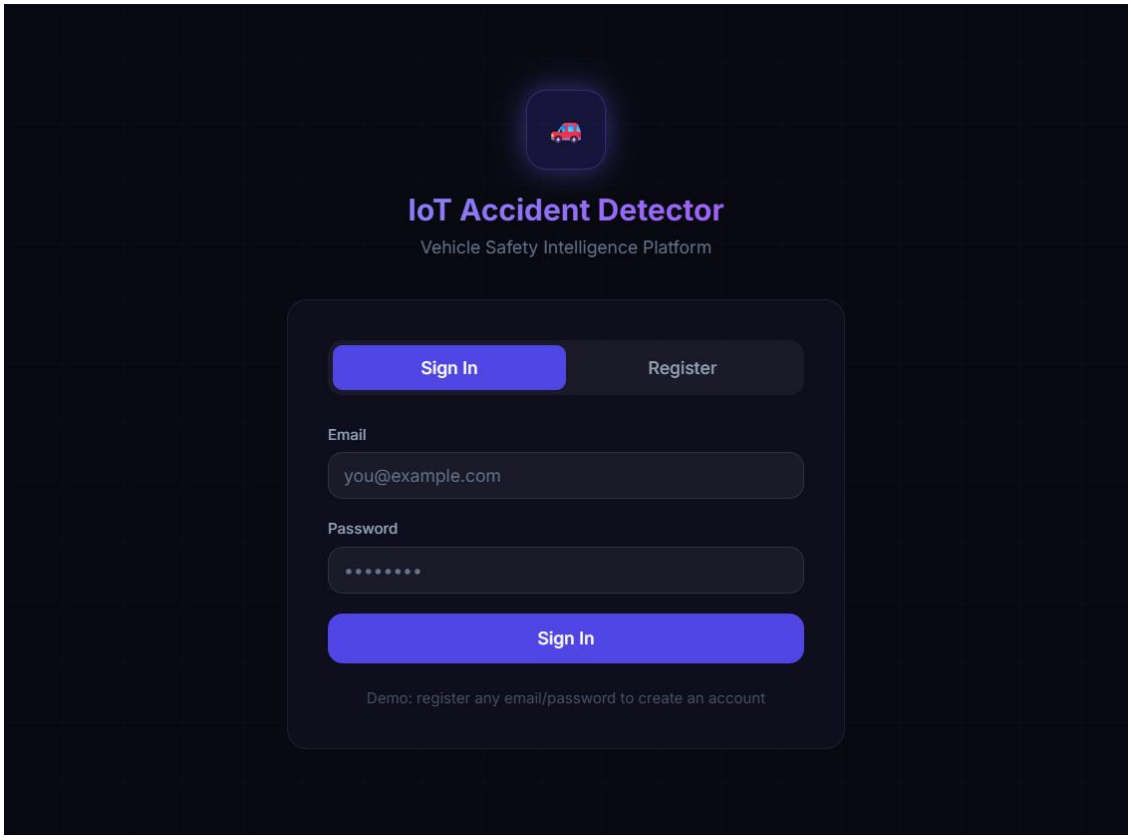


Figure D.1. Local React application sign-in page opened at localhost:5173. This verifies the frontend page is available; it is not an authenticated Fleet Monitor screenshot.

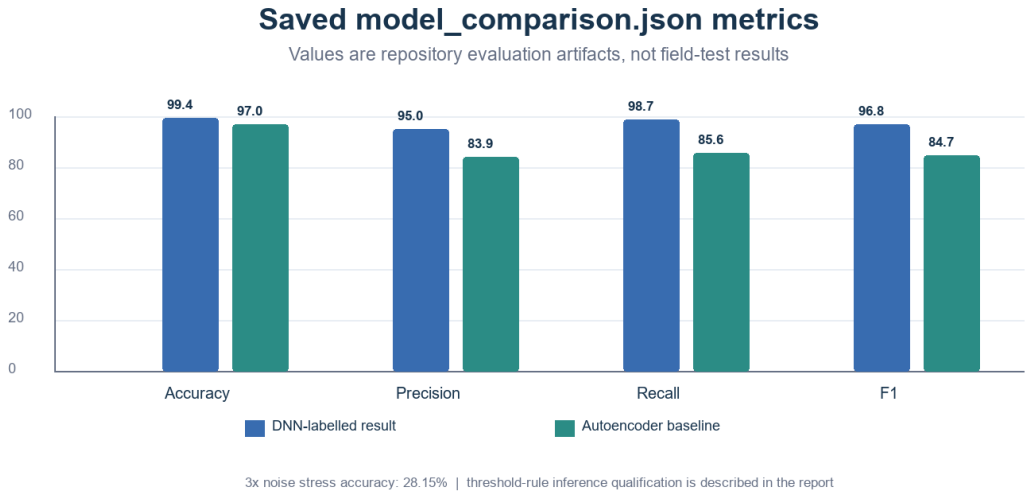


Figure D.2. Metrics and confusion matrices visualized from the saved model_comparison.json artifact. Values are reproduced as stored; the final prediction code currently uses threshold rules rather than a learned DNN forward pass.